

DB实验

姓名： 李冬阳

班级： 计算机2405

学号： 2244113205

数据库：MySQL云数据库

url: jdbc:mysql://8.152.100.169/mydb

username: mydb

password: 123456

一.数据库设计

学号: 2244113205

S205 表设计

- **s_id**: 由表中可知, id 由 8 位数字, 所以直接固定长度使用 **char(8)**。
- **s_name**: 这个变长, 给一个 **varchar(20)** 应该够用。
- **sex**: 只需要分男女, 其实可以直接存 **tinyint(1)**, 但是题目给到了“男女”, 这里得用 **char(1)**。
- **b_date**: 这个无需多言, 直接 **DATE** 类型。
- **height**: 这个得存浮点数, 格式固定, 用 **decimal(3, 2)**。
- **dorm**: 存储数据类型很固定, 但是这个宽泛一下, 给 **varchar(20)** 吧。

— Student table

```
CREATE TABLE S205 (
  s_id CHAR(8) NOT NULL COMMENT '学号',
  s_name VARCHAR(20) NOT NULL COMMENT '姓名',
  sex CHAR(1) NOT NULL COMMENT '性别:男/女',
  b_date DATE NOT NULL COMMENT '出生日期',
  height DECIMAL(3,2) NOT NULL COMMENT '身高,单位:米,如1.75',
  dorm VARCHAR(20) NOT NULL COMMENT '宿舍',
  PRIMARY KEY (s_id),
  CONSTRAINT chk_s205_sex CHECK (sex IN ('男', '女')),
  CONSTRAINT chk_s205_height CHECK (height > 0 AND height < 3)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COMMENT = '学生表';
```

— Insert data into S205

```
INSERT INTO S205 (s_id, s_name, sex, b_date, height, dorm) VALUES
('01032010', '王涛', '男', '2004-04-05', 1.72, '东 6 舍 221'),
('01032023', '孙文', '男', '2005-06-10', 1.80, '东 6 舍 221'),
('01032001', '张晓梅', '女', '2004-11-17', 1.58, '东 1 舍 312'),
('01032005', '刘静', '女', '2004-01-10', 1.63, '东 1 舍 312'),
('01032112', '许澍', '男', '2004-02-20', 1.71, '东 6 舍 221'),
('03031011', '王倩', '女', '2005-09-20', 1.66, '东 2 舍 104'),
('03031014', '赵思扬', '男', '2003-06-06', 1.85, '东 18 舍 421'),
('03031051', '周剑', '男', '2003-05-08', 1.68, '东 18 舍 422'),
('03031009', '田菲', '女', '2004-08-11', 1.60, '东 2 舍 104'),
('03031033', '蔡明明', '男', '2004-03-12', 1.75, '东 18 舍 423'),
('03031056', '曹子衿', '女', '2006-12-15', 1.65, '东 2 舍 305');
```

C205 表设计

- **c_id**: “CS-01”格式比较固定，都是 5 个字符，可以使用 **char(5)**，但是为了保留一定容错空间，这里使用 **varchar(10)**。
- **c_name**: 课程名称长度不固定，使用 **varchar(50)** 应该够用。
- **period**: 学时属于数值型数据，且一般不会超过 256 学时，因此使用 **tinyint unsigned** 存储。
- **credit**: 学分取值范围较小，且不会为负数，因此使用 **tinyint unsigned** 存储。
- **teacher**: 教师姓名类型与学生姓名类似，使用 **varchar(20)** 存储。

— Course table

```
CREATE TABLE C205 (
  c_id VARCHAR(10) NOT NULL COMMENT '课程编号',
  c_name VARCHAR(50) NOT NULL COMMENT '课程名称',
  `period` TINYINT UNSIGNED NOT NULL COMMENT '学时',
  credit TINYINT UNSIGNED NOT NULL COMMENT '学分',
  teacher VARCHAR(20) NOT NULL COMMENT '任课教师',
  PRIMARY KEY (c_id),
  CONSTRAINT chk_c205_period CHECK (period > 0),
  CONSTRAINT chk_c205_credit CHECK (credit > 0)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COMMENT = '课程表';
```

— Insert data into C205

```
INSERT INTO C205 (c_id, c_name, period, credit, teacher) VALUES
('CS-01', '数据结构', 60, 3, '张军'),
('CS-02', '计算机组成原理', 80, 4, '王亚伟'),
('CS-04', '人工智能', 40, 2, '李蕾'),
('CS-05', '深度学习', 40, 2, '崔昀'),
('EE-01', '信号与系统', 60, 3, '张明'),
('EE-02', '数字逻辑电路', 100, 5, '胡海东'),
('EE-03', '光电子学与光子学', 40, 2, '石韬');
```

SC205 表设计

- **s_id**: 学生编号，同学生表中的 **s_id**，类型为 **char(8)**。该字段作为外键，参照学生表的主键。
- **c_id**: 课程编号，同课程表中的 **c_id**，类型为 **varchar(10)**。该字段作为外键，参照课程表的主键。
- **grade**: 成绩预期范围为 0.0 ~ 100.0，需要保留一位小数，因此使用 **decimal(4,1)** 存储。

该表直接使用 **s_id** 和 **c_id** 作为联合主键，同时二者分别作为外键，分别关联学生表和课程表。

– Student-course table

```
CREATE TABLE SC205 (
  s_id CHAR(8) NOT NULL COMMENT '学号',
  c_id VARCHAR(10) NOT NULL COMMENT '课程编号',
  grade DECIMAL(4,1) NULL COMMENT '成绩',
  PRIMARY KEY (s_id, c_id),
  CONSTRAINT fk_sc205_s205
    FOREIGN KEY (s_id) REFERENCES S205 (s_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT fk_sc205_c205
    FOREIGN KEY (c_id) REFERENCES C205 (c_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT chk_sc205_grade CHECK (grade IS NULL OR grade BETWEEN 0.0 AND
100.0)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COMMENT = '选课表';
```

– Insert data into SC205

– The grade of ('01032112', 'CS-05') is blank in the original table, so it is inserted as NULL.

```
INSERT INTO SC205 (s_id, c_id, grade) VALUES
('01032010', 'CS-01', 82.0),
('01032010', 'CS-02', 91.0),
('01032010', 'CS-04', 83.5),
('01032001', 'CS-01', 77.5),
('01032001', 'CS-02', 85.0),
('01032001', 'CS-04', 83.0),
('01032005', 'CS-01', 62.0),
('01032005', 'CS-02', 77.0),
('01032005', 'CS-04', 82.0),
('01032023', 'CS-01', 55.0),
('01032023', 'CS-02', 81.0),
('01032023', 'CS-04', 76.0),
('01032112', 'CS-01', 88.0),
('01032112', 'CS-02', 91.5),
('01032112', 'CS-04', 86.0),
('01032112', 'CS-05', NULL),
('03031033', 'EE-01', 93.0),
('03031033', 'EE-02', 89.0),
('03031009', 'EE-01', 88.0),
('03031009', 'EE-02', 78.5),
('03031011', 'EE-01', 91.0),
('03031011', 'EE-02', 86.0),
('03031051', 'EE-01', 78.0),
('03031051', 'EE-02', 58.0),
('03031014', 'EE-01', 79.0),
```

```
('03031014', 'EE-02', 71.0);
```

二.查询验证

下面是简单的查询验证

	s_id	s_name	sex	b_date	height	dorm
1	01032001	张晓梅	女	2004-11-17	1.58	东 1 舍 312
2	01032005	刘静	女	2004-01-10	1.63	东 1 舍 312
3	01032010	王涛	男	2004-04-05	1.72	东 6 舍 221
4	01032023	孙文	男	2005-06-10	1.80	东 6 舍 221
5	01032112	许澍	男	2004-02-20	1.71	东 6 舍 221
6	03031009	田菲	女	2004-08-11	1.60	东 2 舍 104
7	03031011	王倩	女	2005-09-20	1.66	东 2 舍 104
8	03031014	赵思扬	男	2003-06-06	1.85	东 18 舍 421
9	03031033	蔡明明	男	2004-03-12	1.75	东 18 舍 423
10	03031051	周剑	男	2003-05-08	1.68	东 18 舍 422
11	03031056	曹子衿	女	2006-12-15	1.65	东 2 舍 305

	c_id	c_name	period	credit	teacher
1	CS-01	数据结构	60	3	张军
2	CS-02	计算机组成原理	80	4	王亚伟
3	CS-04	人工智能	40	2	李蕾
4	CS-05	深度学习	40	2	崔昀
5	EE-01	信号与系统	60	3	张明
6	EE-02	数字逻辑电路	100	5	胡海东
7	EE-03	光电子学与光子学	40	2	石韬

	s_id	c_id	grade
1	01032001	CS-01	77.5
2	01032001	CS-02	85.0
3	01032001	CS-04	83.0
4	01032005	CS-01	62.0
5	01032005	CS-02	77.0
6	01032005	CS-04	82.0
7	01032010	CS-01	82.0
8	01032010	CS-02	91.0
9	01032010	CS-04	83.5
10	01032023	CS-01	55.0
11	01032023	CS-02	81.0
12	01032023	CS-04	
13	01032112	CS-01	
14	01032112	CS-02	91.5

查询出结果如上图所示

三.SQL操作

1. 查询操作

- 1.1 查询计算机系（CS）所开课程的课程编号、课程名称及学分数

```
select c_id, c_name, credit from c205
where c_id like 'CS-%';
```

SQL

	c_id	c_name	credit
1	CS-01	数据结构	3
2	CS-02	计算机组成原理	4
3	CS-04	人工智能	2
4	CS-05	深度学习	2

- 1.2 查询未选修课程"CS-05"的男生学号及其已选各课程编号、成绩

```
select s205.s_id, sc205.c_id, sc205.grade from s205 join sc205
on s205.s_id = sc205.s_id where s205.sex = '男' and not exists (
    select * from sc205 sc where sc.s_id = s205.s_id
    and sc.c_id = 'CS-05'
);
```

	s_id	c_id	grade
1	01032010	CS-01	82.0
2	01032010	CS-02	91.0
3	01032010	CS-04	83.5
4	01032023	CS-01	55.0
5	01032023	CS-02	81.0
6	01032023	CS-04	76.0
7	03031014	EE-01	79.0
8	03031014	EE-02	71.0
9	03031033	EE-01	93.0
10	03031033	EE-02	89.0
11	03031051	EE-01	78.0
12	03031051	EE-02	58.0

- 1.3 查询 2005 年~2006 年出生学生的基本信息

```
select * from S205 where b_date >= '2005-01-01' and b_date < '2007-01-01';
```

	s_id	s_name	sex	b_date	height	dorm
1	01032023	孙文	男	2005-06-10	1.80	东 6 舍 221
2	03031011	王倩	女	2005-09-20	1.66	东 2 舍 104
3	03031056	曹子衿	女	2006-12-15	1.65	东 2 舍 305

- 1.4 查询每位学生的学号、学生姓名及其已选修课程的学分总数

方案一：没选课的学生不显示

```
select s205.s_id, s205.s_name, sum(credit) as 'credits' from sc205 left join c205
on sc205.c_id = c205.c_id left join s205
on sc205.s_id = s205.s_id group by s_id;
```

	s_id	s_name	credits
1	01032001	张晓梅	9
2	01032005	刘静	9
3	01032010	王涛	9
4	01032023	孙文	9
5	01032112	许澍	11
6	03031009	田菲	8
7	03031011	王倩	8
8	03031014	赵思扬	8
9	03031033	蔡明明	8
10	03031051	周剑	8

方案二：没选课的学生也显示（学分计为 NULL）

```
SQL
select s205.s_id, s205.s_name, sum(c205.credit) as credits from sc205 right join s205
on sc205.s_id = s205.s_id left join c205
on sc205.c_id = c205.c_id
group by s205.s_id, s205.s_name;
```

	s_id	s_name	credits
1	01032001	张晓梅	9
2	01032005	刘静	9
3	01032010	王涛	9
4	01032023	孙文	9
5	01032112	许澍	11
6	03031009	田菲	8
7	03031011	王倩	8
8	03031014	赵思扬	8
9	03031033	蔡明明	8
10	03031051	周剑	8
11	03031056	曹子衿	<null>

- 1.5 查询选修课程"CS-01"的学生中成绩第二高的学生学号

方案一：简单写法（无法处理并列第一）

```
SQL
select * from sc205 where c_id = 'CS-01'
order by grade desc limit 1 offset 1;
```


	s_id	c_id	grade
1	01032010	CS-01	82.0

方案二：处理并列第一的情况

SQL

```
select s_id from sc205 where c_id = 'CS-01' and grade = (
    select distinct grade from sc205 where c_id = 'CS-01'
    order by grade desc limit 1 offset 1
);
```

- 1.6 查询平均成绩低于"许澍"同学的学生学号、姓名和平均成绩，并按学号降序排列

SQL

```
select s205.s_id,s205.s_name,avg(sc205.grade) from s205 join sc205 on s205.s_id =
sc205.s_id
    group by s205.s_id having avg(sc205.grade) < (
        select avg(sc205.grade) from s205 join sc205 on s205.s_id = sc205.s_id
            where s205.s_name = '许澍'
    ) order by s205.s_id desc ;
```

	s_id	s_name	`avg(sc205.grade)`
1	03031051	周剑	68.00000
2	03031014	赵思扬	75.00000
3	03031009	田菲	83.25000
4	01032023	孙文	70.66667
5	01032010	王涛	85.50000
6	01032005	刘静	73.66667
7	01032001	张晓梅	81.83333

- 1.7 查询选修了计算机专业全部课程（课程编号为"CS-xx"）的学生姓名及已获得的学分总数

SQL

```
select s205.s_name, t.total_credit from s205 join (
    select sc205.s_id, sum(c205.credit) total_credit from sc205 join c205
on sc205.c_id = c205.c_id where sc205.c_id like 'CS-%' group by sc205.s_id
    having count(distinct sc205.c_id) = (
        select count(*) from c205 where c205.c_id like 'CS-%'
    )
) t on s205.s_id = t.s_id;
```

	s_name	total_credit
1	许澍	11

- 1.8 查询选修了 3 门以上课程（包括 3 门）的学生中平均成绩最高的同学学号及姓名

```
SQL
select s205.s_id, s205.s_name from s205 join (
    select sc205.s_id, avg(grade) avg_grade, count(*) num from sc205 group by sc205.s_id
    having num >= 3) t on t.s_id = s205.s_id
order by t.avg_grade desc limit 1;
```

	s_id	s_name
1	01032112	许澍

2. 插入记录

分别在 S205 和 C205 表中加入记录 ('01032005', '刘竞', '男', '2003-12-10', 1.75, '东 14 舍 312') 及 ('CS-03', '离散数学', 64, 4, '陈建明')。

```
SQL
insert into s205
values ('01032005', '刘竞', '男', '2003-12-10', 1.75, '东 14 舍 312');
insert into c205
values ('CS-03', '离散数学', 64, 4, '陈建明');
```

结果,第一个insert报错,原因在于设置id为主键
第二个insert正常执行

```
[2026-06-30 10:24:13] [23000][1062] Duplicate entry '01032005' for key
's205.PRIMARY'
[2026-06-30 10:24:36] 67 ms 中有 1 行受到影响
```

3. 删除记录

将 S205 表中已修学分数大于 20 的学生记录删除。

SQL

```
delete s205.sc205 from s205 join sc205 on s205.s_id = sc205.s_id
join (select * from sc205 join c205 on sc205.c_id = c205.c_id
      group by sc205.s_id having sum(credit) > 20) t
on s205.s_id = t.s_id;
```

[2026-06-30 10:28:36] mydb> delete s205,sc205 from s205 join sc205 on s205.s_id = sc205.s_id

```
join (select sc205.s_id from sc205 join c205 on sc205.c_id = c205.c_id
      group by sc205.s_id having sum(credit) > 20) t
on s205.s_id = t.s_id
```

[2026-06-30 10:28:36] 在 65 ms 内完成

4. 更新记录

将"张明"老师负责的"数字电子技术"课程的学时数调整为 36，同时增加一个学分。

SQL

```
update c205 set `period`=36,credit=credit+1
where c_name = '数字电子技术' and teacher = '张明'
```

5. 创建视图

- 5.1 居住在"东 18 舍"的男生视图，包括学号、姓名、出生日期、身高等属性

SQL

```
create view v_east18_s205 as
select * from s205 where sex='男' and dorm like '东 18 舍%';
```

	s_id	s_name	sex	b_date	height	dorm
1	03031014	赵思扬	男	2003-06-06	1.85	东 18 舍 421
2	03031033	蔡明明	男	2004-03-12	1.75	东 18 舍 423
3	03031051	周剑	男	2003-05-08	1.68	东 18 舍 422

- 5.2 "张明"老师所开设课程情况的视图，包括课程编号、课程名称、平均成绩等属性

SQL

```
create view v_zhangming_avg_grade as
select sc205.c_id,t.c_name,avg(grade) avg_grade
from sc205 join (select c_id,c_name from c205 where teacher='张明') t
on sc205.c_id = t.c_id group by sc205.c_id;
```

	c_id	c_name	avg_grade
1	EE-01	信号与系统	85.80000

- 5.3 所有选修了"人工智能"课程的学生视图，包括学号、姓名、成绩等属性

SQL

```
create view v_ai_class_stu as
select s205.s_id, s205.s_name, grade from s205 join (
    select sc205.s_id, sc205.grade from sc205 where c_id in (
        select c205.c_id from c205 where c_name = '人工智能')
) t on t.s_id = s205.s_id;
```

	s_id	s_name	grade
1	01032001	张晓梅	83.0
2	01032005	刘静	82.0
3	01032010	王涛	83.5
4	01032023	孙文	76.0
5	01032112	许澍	86.0

	Tables_in_mydb	Table_type
1	c205	BASE TABLE
2	s205	BASE TABLE
3	sc205	BASE TABLE
4	v_ai_class_stu	VIEW
5	v_east18_s205	VIEW
6	v_zhangming_avg_grade	VIEW

四.程序设计

数据生成

- 课程表: 使用python进行爬取教务处
- 学生表: 源数据难以爬取,采用 从一些课程群,竞赛群 找到一些已知名单+py数据处理; 身高和宿舍信息均为生成;但最终只获得1700条数据,学号和姓名同时存在的表过少
- 选课表: 无法爬虫获取,采用程序随机生成

以上所有代码和数据文件见附件

主要使用为python脚本+Agent数据处理

S205: s_id 最大长度 10, 超过 CHAR(8)
SC205: s_id 最大长度 10, 超过 CHAR(8)
C205: 字段长度、主键、period、credit 都满足
所以需要数据库进行修改

SQL

```
ALTER TABLE SC205 DROP FOREIGN KEY fk_sc205_s205;

ALTER TABLE S205 MODIFY s_id CHAR(10) NOT NULL COMMENT '学号';
ALTER TABLE SC205 MODIFY s_id CHAR(10) NOT NULL COMMENT '学号';

UPDATE S205
SET s_id = RPAD(TRIM(s_id), 10, '0')
WHERE CHAR_LENGTH(TRIM(s_id)) < 10;

UPDATE SC205
SET s_id = RPAD(TRIM(s_id), 10, '0')
WHERE CHAR_LENGTH(TRIM(s_id)) < 10;

ALTER TABLE SC205
ADD CONSTRAINT fk_sc205_s205
FOREIGN KEY (s_id) REFERENCES S205 (s_id)
ON UPDATE CASCADE
ON DELETE RESTRICT;
```

修改数据库,把当前存在数据直接末尾补 0

多线程增删数据

在本程序中,使用SpringBoot框架,采取了一个web页面,把一些操作放在web页面上,方便进行使用.

数据库实验数据导入控制台

S205 / C205 / SC205 数据集导入与对比

第一问 Q1 第二问 Q2

S205
1,000 行

C205
100 行

SC205
20,000 行

S205
1,000 行, 6 列

s_id s_name sex b_date height dorm

2206312141	傅浩思	男	2005-09-13	1.82	东1舍 706
2215112645	卡阳程	男	2005-06-20	1.89	西18舍 219
2216515598	安柳欣	女	2005-06-11	1.68	西14舍 712
2221110595	吴梓浩	男	2005-02-12	1.90	东16舍 703
2221313184	朱奕佑	男	2005-12-04	1.86	东13舍 524
2221313210	张雨安	男	2005-11-12	1.78	东13舍 630
2221313309	姜佑鹏	女	2005-05-02	1.56	西3舍 711
2221323212	谢凤轩	男	2005-06-04	1.89	东2舍 821
2221323227	梅正航	女	2005-04-23	1.69	东1舍 110
2221411131	唐怡扬	女	2005-03-10	1.73	东2舍 502

直接写入 多线程写入 恢复基线

操作结果

已加载 第一问 Q1。

重点代码记录

普通串行

```

private void insertS205(Connection connection, List<List<String>> rows) throws
SQLException {
    try (PreparedStatement statement = connection.prepareStatement(
        "INSERT INTO S205 (s_id, s_name, sex, b_date, height, dorm) VALUES (?, ?, ?,
        ?, ?, ?)")) {
        for (List<String> row : rows) {
            statement.setString(1, row.get(0));
            statement.setString(2, row.get(1));
            statement.setString(3, row.get(2));
            statement.setDate(4, Date.valueOf(row.get(3)));
            statement.setBigDecimal(5, new BigDecimal(row.get(4)));
            statement.setString(6, row.get(5));
            statement.addBatch();
        }
        statement.executeBatch();
    }
}

```

```

private void insertC205(Connection connection, List<List<String>> rows) throws
SQLException {
    try (PreparedStatement statement = connection.prepareStatement(
        "INSERT INTO C205 (c_id, c_name, `period`, credit, teacher) VALUES (?, ?, ?,
        ?, ?)")) {
        for (List<String> row : rows) {
            statement.setString(1, row.get(0));
            statement.setString(2, row.get(1));
            statement.setInt(3, Integer.parseInt(row.get(2)));
            statement.setInt(4, Integer.parseInt(row.get(3)));
            statement.setString(5, row.get(4));
            statement.addBatch();
        }
        statement.executeBatch();
    }
}

```

```

private void insertSc205(Connection connection, List<List<String>> rows) throws
SQLException {
    try (PreparedStatement statement = connection.prepareStatement(
        "INSERT INTO SC205 (s_id, c_id, grade) VALUES (?, ?, ?)")) {
        for (List<String> row : rows) {
            statement.setString(1, row.get(0));
            statement.setString(2, row.get(1));
            if (row.get(2).isBlank()) {
                statement.setNull(3, Types.DECIMAL);
            } else {
                statement.setBigDecimal(3, new BigDecimal(row.get(2)));
            }
            statement.addBatch();
        }
    }
}

```

```

    }
    statement.executeBatch();
}
}

```

多线程

JAVA

```

private int defaultThreadCount() {
    return Math.max(2, Math.min(8, Runtime.getRuntime().availableProcessors()));
}

private void runChunks(List<List<String>> rows, int threadCount, ChunkOperation
operation)
    throws InterruptedException, ExecutionException {
    ExecutorService executorService = Executors.newFixedThreadPool(threadCount);
    try {
        List<Future<Void>> futures = new ArrayList<>();
        for (int start = 0; start < rows.size(); start += BATCH_SIZE) {
            int end = Math.min(start + BATCH_SIZE, rows.size());
            List<List<String>> chunk = List.copyOf(rows.subList(start, end));
            futures.add(executorService.submit(new Callable<Void>() {
                @Override
                public Void call() throws Exception {
                    operation.apply(chunk);
                    return null;
                }
            }));
        }
        for (Future<Void> future : futures) {
            future.get();
        }
    } finally {
        executorService.shutdown();
    }
}

```

执行结果

第一问 的插入语句

操作结果

```
执行结果
数据集: 第一问 Q1
模式: 直接写入
S205 写入行数: 1,000
C205 写入行数: 100
SC205 写入行数: 20,000
线程数: 1
耗时(ms): 956
说明: Import completed
```

操作结果

```
执行结果
数据集: 第一问 Q1
模式: 多线程写入
S205 写入行数: 1,000
C205 写入行数: 100
SC205 写入行数: 20,000
线程数: 8
耗时(ms): 1,075
说明: Import completed
```

实验发现,在少量数据时,多线程耗时和单线程差不多,甚至更慢.
原因可能在于网络IO时间占主导因素

第二问 的插入语句

操作结果

```
执行结果
数据集: 第二问 Q2
模式: 直接写入
S205 写入行数: 5,000
C205 写入行数: 1,000
SC205 写入行数: 200,000
线程数: 1
耗时(ms): 16,352
说明: Import completed
```

操作结果

```
执行结果
数据集：第二问 Q2
模式：多线程写入
S205 写入行数：5,000
C205 写入行数：1,000
SC205 写入行数：200,000
线程数：8
耗时(ms)：6,464
说明：Import completed
```

速度明显快很多

小插曲

添加数据库	root密码	远程数据库	phpMyAdmin	高级设置	同步数据库	回收站	需求反馈	全部分类	请输入数据库名	Q	刷新	设置
☐	数据库名	用户名	密码	容量	备份	数据库位置	备注	操作				
☐	mydb	mydb	*****	未配置	点击备份 导入	本地数据库	mydb	管理	权限	工具	改密	删除
☐	rust_db	rust_db	*****	未配置	点击备份 导入	本地数据库	rust_db	管理	权限	工具	改密	删除
☐	msgcenter	msgcenter	*****	未配置	点击备份 导入	本地数据库	msgcenter	管理	权限	工具	改密	删除
☐	xjtuhub	xjtuhub	*****	未配置	点击备份 导入	本地数据库	xjtuhub	管理	权限	工具	改密	删除
☐	jxt	jxt	*****	未配置	点击备份 导入	本地数据库	jxt	管理	权限	工具	改密	删除
☐	kfc	kfc	*****	未配置	点击备份 导入	本地数据库	kfc	管理	权限	工具	改密	删除
☐	hmdp	hmdp	*****	未配置	点击备份 导入	本地数据库	hmdp	管理	权限	工具	改密	删除
☐	smart_care	smart_care	*****	未配置	点击备份 导入	本地数据库	smart_care	管理	权限	工具	改密	删除
☐	lease	lease	*****	未配置	点击备份 导入	本地数据库	lease	管理	权限	工具	改密	删除
☐	请选择批量操作											
	批量操作											
										< 1 >	10条/页	共 9 条 前往 1

这是我的服务器的数据库,在执行多线程时候同时点击了两个按钮,写入的时候把Mysql给打爆了,最后还得手动重启Mysql

删除

第一问要求在 SC205 中随机删除 200 条成绩低于 60 分或成绩为 NULL 的选课记录。这里完全使用 SQL 完成，先把随机选中的 200 条主键保存到临时表，再通过 DELETE JOIN 删除。

```

START TRANSACTION;

CREATE TEMPORARY TABLE tmp_delete_sc205 AS
SELECT s_id, c_id
FROM SC205
WHERE grade < 60 OR grade IS NULL
ORDER BY RAND()
LIMIT 200;

DELETE sc
FROM SC205 sc
JOIN tmp_delete_sc205 t
  ON sc.s_id = t.s_id
  AND sc.c_id = t.c_id;

SELECT ROW_COUNT() AS deleted_rows;

DROP TEMPORARY TABLE tmp_delete_sc205;

COMMIT;

```

说明：

1. `WHERE grade < 60 OR grade IS NULL` 对应题目要求中的“成绩低于 60 分（含 NULL）”。
2. `ORDER BY RAND() LIMIT 200` 用 SQL 随机选出 200 条候选选课记录。
3. 先保存到临时表，是为了固定本次删除的 200 条记录，避免删除过程中重新随机。
4. 删除条件使用 `(s_id, c_id)`，因为 `SC205` 的主键是 `(s_id, c_id)`，可以精确定位一条选课记录。
5. 放在事务中执行，保证临时表选择和删除操作作为一个整体完成。

如果要先确认候选记录数量是否足够，可以执行：

```

SELECT COUNT(*) AS low_grade_count
FROM SC205
WHERE grade < 60 OR grade IS NULL;

```

	low_grade_count
1	2245

删除完成后，可以验证剩余记录数：

```
SELECT COUNT(*) AS sc205_rows_after_delete
FROM SC205;
```

	sc205_rows_after_delete	
1	19800	

第一问中 **SC205** 原始补充到约 20000 行，删除 200 行后应保留 19800 行

查询

第二问要求将数据量扩大到 **S205** 约 5000 行、**C205** 约 1000 行、**SC205** 约 200000 行，并对第三部分中的部分查询写出不少于 3 个不同 SQL，分析其运行效率。以下内容只使用 SQL 完成。

补测说明：补充执行计划时，当前数据库中实际数据量为 **S205=1000**、**C205=100**、**SC205=19800**。因此下面补充的 **EXPLAIN ANALYZE** 数值以当前库为准；第二问 20 万行数据下，受影响的位置相同，但耗时差距会被进一步放大。

第二问数据导入 SQL

检查数据情况

```
SELECT 'S205' AS table_name, COUNT(*) AS row_count FROM S205
UNION ALL
SELECT 'C205', COUNT(*) FROM C205
UNION ALL
SELECT 'SC205', COUNT(*) FROM SC205;
```

```
S205  5000
C205  1000
SC205 200000
```

查询 1：每位学生已选修课程的学分总数

对应第三部分 1.4。

写法一：直接连接三张表后分组。

```
EXPLAIN ANALYZE
SELECT s.s_id, s.s_name, SUM(c.credit) AS total_credit
FROM S205 s
JOIN SC205 sc ON sc.s_id = s.s_id
JOIN C205 c ON c.c_id = sc.c_id
GROUP BY s.s_id, s.s_name;
```

```
-> Table scan on <temporary> (actual time=164..165 rows=1000 loops=1)
-> Aggregate using temporary table (actual time=164..164 rows=1000 loops=1)
-> Nested loop inner join (cost=734 rows=897) (actual time=0.0724..109 rows=19800 loops=1)
-> Nested loop inner join (cost=420 rows=897) (actual time=0.0613..46.7 rows=19800 loops=1)
-> Table scan on s (cost=106 rows=897) (actual time=0.0404..1.25 rows=1000 loops=1)
-> Covering index lookup on sc using PRIMARY (s_id=s.s_id) (cost=0.25 rows=1) (actual time=0.0000..0.0000 rows=1 loops=1)
-> Single-row index lookup on c using PRIMARY (c_id=sc.c_id) (cost=0.25 rows=1) (actual time=0.0000..0.0000 rows=1 loops=1)
```

写法二：先在 **SC205** 和 **C205** 上聚合，再连接 **S205**。

```
EXPLAIN ANALYZE
SELECT s.s_id, s.s_name, t.total_credit
FROM S205 s
JOIN (
  SELECT sc.s_id, SUM(c.credit) AS total_credit
  FROM SC205 sc
  JOIN C205 c ON c.c_id = sc.c_id
  GROUP BY sc.s_id
) t ON t.s_id = s.s_id;
```

```
-> Nested loop inner join (cost=338 rows=0) (actual time=17.7..19.1 rows=1000 loops=1)
-> Table scan on t (cost=2.5..2.5 rows=0) (actual time=17.7..17.8 rows=1000 loops=1)
-> Materialize (cost=0..0 rows=0) (actual time=17.7..17.7 rows=1000 loops=1)
-> Table scan on <temporary> (actual time=17.5..17.6 rows=1000 loops=1)
-> Aggregate using temporary table (actual time=17.5..17.5 rows=1000 loops=1)
-> Nested loop inner join (cost=174 rows=1345) (actual time=0.0645..9.15 rows=19800 loops=1)
-> Table scan on c (cost=10.2 rows=100) (actual time=0.0413..0.0682 rows=100 loops=1)
-> Covering index lookup on sc using fk_sc205_c205 (c_id=c.c_id) (cost=0.31 rows=1) (actual time=0.0000..0.0000 rows=1 loops=1)
-> Single-row index lookup on s using PRIMARY (s_id=t.s_id) (cost=0.25 rows=1) (actual time=0.0000..0.0000 rows=1 loops=1)
```

写法三：保留没有选课的学生，未选课学生总学分显示为 0。

```
EXPLAIN ANALYZE
SELECT s.s_id, s.s_name, COALESCE(SUM(c.credit), 0) AS total_credit
FROM S205 s
LEFT JOIN SC205 sc ON sc.s_id = s.s_id
LEFT JOIN C205 c ON c.c_id = sc.c_id
GROUP BY s.s_id, s.s_name;
```

```
-> Table scan on <temporary> (actual time=59.7..60 rows=1000 loops=1)
  -> Aggregate using temporary table (actual time=59.7..59.7 rows=1000 loops=1)
    -> Nested loop left join (cost=734 rows=897) (actual time=0.0674..40.7 rows=19800 loops=1)
      -> Nested loop left join (cost=420 rows=897) (actual time=0.0588..16.4 rows=19800 loops=1)
        -> Table scan on s (cost=106 rows=897) (actual time=0.0387..0.372 rows=1000 loops=1)
          -> Covering index lookup on sc using PRIMARY (s_id=s.s_id) (cost=0.25 rows=1) (actual time=0.0000..0.0000 rows=1 loops=897)
            -> Single-row index lookup on c using PRIMARY (c_id=sc.c_id) (cost=0.25 rows=1) (actual time=0.0000..0.0000 rows=1 loops=897)
```

优化索引:

```
CREATE INDEX idx_sc205_sid_cid ON SC205 (s_id, c_id);
```

三个方法的执行计划为:

方法一:

```
-> Table scan on <temporary> (actual time=61.4..61.6 rows=1000 loops=1)
  -> Aggregate using temporary table (actual time=61.4..61.4 rows=1000 loops=1)
    -> Nested loop inner join (cost=8508 rows=17984) (actual time=0.0776..41.5 rows=1980)
      -> Nested loop inner join (cost=2214 rows=17984) (actual time=0.067..15.5 rows=1980)
        -> Table scan on s (cost=106 rows=897) (actual time=0.0392..0.379 rows=1000 loops=1)
        -> Covering index lookup on sc using PRIMARY (s_id=s.s_id) (cost=0.347 rows=20)
        -> Single-row index lookup on c using PRIMARY (c_id=sc.c_id) (cost=0.25 rows=1) (actual time=0.000..0.001 rows=1 loops=1)
```

方法二:

```
-> Nested loop inner join (cost=92829 rows=878163) (actual time=42.3..44.6 rows=1000 loops=1)
  -> Table scan on s (cost=106 rows=897) (actual time=0.0383..0.326 rows=1000 loops=1)
  -> Index lookup on t using <auto_key0> (s_id=s.s_id) (cost=11134..11139 rows=21.9) (actual time=0.000..0.001 rows=1 loops=1)
    -> Materialize (cost=11134..11134 rows=979) (actual time=42.2..42.2 rows=1000 loops=1)
      -> Group aggregate: sum(c.credit) (cost=11036 rows=979) (actual time=0.0779..39.6 rows=1000 loops=1)
        -> Nested loop inner join (cost=9073 rows=19628) (actual time=0.0417..33.6 rows=1980)
          -> Covering index scan on sc using PRIMARY (cost=2203 rows=19628) (actual time=0.000..0.001 rows=1 loops=1)
          -> Single-row index lookup on c using PRIMARY (c_id=sc.c_id) (cost=0.25 rows=1) (actual time=0.000..0.001 rows=1 loops=1)
```

方法三:

```
-> Table scan on <temporary> (actual time=63.6..63.8 rows=1000 loops=1)
  -> Aggregate using temporary table (actual time=63.6..63.6 rows=1000 loops=1)
    -> Nested loop left join (cost=8508 rows=17984) (actual time=0.0651..44 rows=19800 loops=1)
      -> Nested loop left join (cost=2214 rows=17984) (actual time=0.056..16.7 rows=19800 loops=1)
        -> Table scan on s (cost=106 rows=897) (actual time=0.0348..0.422 rows=1000 loops=1)
        -> Covering index lookup on sc using PRIMARY (s_id=s.s_id) (cost=0.347 rows=20)
        -> Single-row index lookup on c using PRIMARY (c_id=sc.c_id) (cost=0.25 rows=1) (actual time=0.000..0.001 rows=1 loops=1)
```

效率分析:

写法一会先连接三张表，再按学生分组；在 **SC205** 有 200000 行时，中间结果较大。写

法二先按 **s_id** 聚合，临时结果最多约 5000 行，再连接学生表，通常中间结果更小。

写法三使用 **LEFT JOIN**，语义更完整，但需要保留所有学生记录，执行代价可能略高。

idx_sc205_sid_cid 能帮助按学生连接和分组。

查询 2：平均成绩低于“许澍”的学生

对应第三部分 1.6。

写法一：在 **HAVING** 中使用标量子查询。

```

EXPLAIN ANALYZE
SELECT s.s_id, s.s_name, AVG(sc.grade) AS avg_grade
FROM S205 s
JOIN SC205 sc ON sc.s_id = s.s_id
GROUP BY s.s_id, s.s_name
HAVING AVG(sc.grade) < (
    SELECT AVG(sc2.grade)
    FROM S205 s2
    JOIN SC205 sc2 ON sc2.s_id = s2.s_id
    WHERE s2.s_name = '许澍'
)
ORDER BY s.s_id DESC;

```

写法二：先求所有学生平均分，再与目标学生平均分比较。

```

EXPLAIN ANALYZE
WITH avg_table AS (
    SELECT s.s_id, s.s_name, AVG(sc.grade) AS avg_grade
    FROM S205 s
    JOIN SC205 sc ON sc.s_id = s.s_id
    WHERE sc.grade IS NOT NULL
    GROUP BY s.s_id, s.s_name
),
target_avg AS (
    SELECT avg_grade
    FROM avg_table
    WHERE s_name = '许澍'
    LIMIT 1
)
SELECT a.s_id, a.s_name, a.avg_grade
FROM avg_table a
CROSS JOIN target_avg t
WHERE a.avg_grade < t.avg_grade
ORDER BY a.s_id DESC;

```

写法三：把目标学生平均分先单独聚合成派生表。


```

EXPLAIN ANALYZE
SELECT a.s_id, a.s_name, a.avg_grade
FROM (
    SELECT s.s_id, s.s_name, AVG(sc.grade) AS avg_grade
    FROM S205 s
    JOIN SC205 sc ON sc.s_id = s.s_id
    WHERE sc.grade IS NOT NULL
    GROUP BY s.s_id, s.s_name
) a
JOIN (
    SELECT AVG(sc.grade) AS target_avg
    FROM S205 s
    JOIN SC205 sc ON sc.s_id = s.s_id
    WHERE s.s_name = '许澍'
    AND sc.grade IS NOT NULL
) t
WHERE a.avg_grade < t.target_avg
ORDER BY a.s_id DESC;

```

优化索引:

```

CREATE INDEX idx_s205_name_sid ON S205 (s_name, s_id);
CREATE INDEX idx_sc205_sid_grade ON SC205 (s_id, grade);

```

未建立优化索引时, 写法一的关键执行计划如下:

```

-> Sort: s.s_id DESC, s.s_name (actual time=34.8..34.8 rows=214 loops=1)
-> Filter: (avg(sc.grade) < (select #2)) (actual time=34.2..34.7 rows=214 loops=1)
-> Aggregate using temporary table (actual time=33.8..33.8 rows=1000 loops=1)
-> Nested loop inner join (actual time=0.0557..1.14 rows=19800 loops=1)
-> Table scan on s (actual time=0.0342..0.342 rows=1000 loops=1)
-> Index lookup on sc using PRIMARY (s_id=s.s_id)
-> Select #2
-> Aggregate: avg(sc2.grade) (actual time=0.376..0.377 rows=1 loops=1)
-> Filter: (s2.s_name = _utf8mb4'傅清思')
-> Table scan on s2 (actual time=0.0306..0.268 rows=1000 loops=1)

```

建立索引后, 写法一的关键执行计划如下:

```

-> Sort: s.s_id DESC, s.s_name (actual time=42.9..42.9 rows=214 loops=1)
-> Filter: (avg(sc.grade) < (select #2)) (actual time=42.2..42.7 rows=214 loops=1)
-> Aggregate using temporary table (actual time=42.1..42.1 rows=1000 loops=1)
-> Nested loop inner join (actual time=0.0703..1.8 rows=19800 loops=1)
    -> Covering index scan on s using idx_s205_name_sid
    -> Index lookup on sc using PRIMARY (s_id=s.s_id)
-> Select #2
    -> Aggregate: avg(sc2.grade) (actual time=0.0506..0.0507 rows=1 loops=1)
    -> Covering index lookup on s2 using idx_s205_name_sid
        (s_name=_utf8mb4'傅清思')
    -> Index lookup on sc2 using PRIMARY (s_id=s2.s_id)

```

写法三在建立索引前后的关键变化如下：

```

-- 建立索引前
-> Sort: a.s_id DESC (actual time=32.6..32.6 rows=214 loops=1)
-> Materialize (actual time=32.3..32.3 rows=1000 loops=1)
-> Aggregate using temporary table (actual time=31.6..31.6 rows=1000 loops=1)
-> Nested loop inner join (actual time=0.0382..15.1 rows=19167 loops=1)
    -> Table scan on s
    -> Index lookup on sc using PRIMARY (s_id=s.s_id)

-- 建立索引后
-> Sort: a.s_id DESC (actual time=33.1..33.2 rows=214 loops=1)
-> Materialize (actual time=32.8..32.8 rows=1000 loops=1)
-> Aggregate using temporary table (actual time=32.1..32.1 rows=1000 loops=1)
-> Nested loop inner join (actual time=0.115..15.8 rows=19167 loops=1)
    -> Covering index scan on s using idx_s205_name_sid
    -> Index lookup on sc using PRIMARY (s_id=s.s_id)

```

效率分析：

`idx_s205_name_sid` 的主要影响点在目标学生子查询：由 `Table scan on s2` 变为 `Covering index lookup on s2 using idx_s205_name_sid`，目标学生平均分子查询从约 `0.376 ms` 降到约 `0.0507 ms`。但是整个查询仍然要对所有学生的 `SC205` 成绩做聚合，主耗时在 `19800` 行选课记录扫描和临时表聚合上，所以总耗时不一定下降。`idx_sc205_sid_grade` 对本查询的影响不明显，因为原主键 `(s_id, c_id)` 已经能按 `s_id` 找到某学生的选课记录。

查询 3：选修 3 门课程且平均成绩最高的学生

对应第三部分 1.8。

写法一：只返回平均分最高的一名学生。

```
EXPLAIN ANALYZE
SELECT s.s_id, s.s_name, t.avg_grade
FROM S205 s
JOIN (
    SELECT sc.s_id, AVG(sc.grade) AS avg_grade, COUNT(*) AS course_count
    FROM SC205 sc
    WHERE sc.grade IS NOT NULL
    GROUP BY sc.s_id
    HAVING COUNT(*) >= 3
) t ON t.s_id = s.s_id
ORDER BY t.avg_grade DESC
LIMIT 1;
```

写法二：使用窗口函数，返回平均分并列第一的所有学生。

```
EXPLAIN ANALYZE
WITH ranked_students AS (
    SELECT
        sc.s_id,
        AVG(sc.grade) AS avg_grade,
        COUNT(*) AS course_count,
        DENSE_RANK() OVER (ORDER BY AVG(sc.grade) DESC) AS grade_rank
    FROM SC205 sc
    WHERE sc.grade IS NOT NULL
    GROUP BY sc.s_id
    HAVING COUNT(*) >= 3
)
SELECT s.s_id, s.s_name, r.avg_grade
FROM ranked_students r
JOIN S205 s ON s.s_id = r.s_id
WHERE r.grade_rank = 1;
```

写法三：先求最高平均分，再回连聚合结果。

```

EXPLAIN ANALYZE
WITH student_avg AS (
    SELECT sc.s_id, AVG(sc.grade) AS avg_grade, COUNT(*) AS course_count
    FROM SC205 sc
    WHERE sc.grade IS NOT NULL
    GROUP BY sc.s_id
    HAVING COUNT(*) >= 3
),
max_avg AS (
    SELECT MAX(avg_grade) AS max_grade
    FROM student_avg
)
SELECT s.s_id, s.s_name, a.avg_grade
FROM student_avg a
JOIN max_avg m ON a.avg_grade = m.max_grade
JOIN S205 s ON s.s_id = a.s_id;

```

优化索引：

```
CREATE INDEX idx_sc205_sid_grade ON SC205 (s_id, grade);
```

未建立 `idx_sc205_sid_grade` 时，三种写法的关键执行计划如下：

```

-- 写法一
-> Limit: 1 row(s) (actual time=11.4..11.4 rows=1 loops=1)
    -> Sort: t.avg_grade DESC (actual time=11.4..11.4 rows=1 loops=1)
        -> Materialize (actual time=11..11 rows=1000 loops=1)
            -> Group aggregate (actual time=0.0497..10.6 rows=1000 loops=1)
                -> Filter: (sc.grade is not null)
                    -> Index scan on sc using PRIMARY
                        (actual time=0.034..4.85 rows=19800 loops=1)

-- 写法二
-> Nested loop inner join (actual time=15.7..15.8 rows=1 loops=1)
    -> Window aggregate: dense_rank() (actual time=14.9..15.5 rows=1000 loops=1)
        -> Sort: avg_grade DESC (actual time=14.9..15 rows=1000 loops=1)
            -> Aggregate using temporary table (actual time=14.4..14.4 rows=1000 loops=1)
                -> Table scan on sc (actual time=0.0343..5.26 rows=19800 loops=1)

-- 写法三
-> Nested loop inner join (actual time=0.0237..0.186 rows=1 loops=1)
    -> Filter: (a.avg_grade = '84.94706') (actual time=0.0108..0.173 rows=1 loops=1)
        -> Table scan on a (actual time=0.00396..0.0855 rows=1000 loops=1)

```

建立 `idx_sc205_sid_grade` 后，三种写法的关键执行计划如下：

```
-- 写法一
-> Limit: 1 row(s) (actual time=10.7..10.7 rows=1 loops=1)
    -> Sort: t.avg_grade DESC (actual time=10.7..10.7 rows=1 loops=1)
        -> Materialize (actual time=10.3..10.3 rows=1000 loops=1)
            -> Group aggregate (actual time=0.0599..9.85 rows=1000 loops=1)
                -> Filter: (sc.grade is not null)
                    -> Covering index scan on sc using idx_sc205_sid_grade
                        (actual time=0.0433..4.24 rows=19800 loops=1)

-- 写法二
-> Nested loop inner join (actual time=14.2..14.3 rows=1 loops=1)
    -> Window aggregate: dense_rank() (actual time=13.4..14 rows=1000 loops=1)
        -> Sort: avg_grade DESC (actual time=13.4..13.5 rows=1000 loops=1)
            -> Aggregate using temporary table (actual time=12.9..12.9 rows=1000
loops=1)
                -> Covering index scan on sc using idx_sc205_sid_grade
                    (actual time=0.0304..4.29 rows=19800 loops=1)

-- 写法三
-> Nested loop inner join (actual time=0.028..0.219 rows=1 loops=1)
    -> Filter: (a.avg_grade = '84.94706') (actual time=0.0105..0.201 rows=1 loops=1)
        -> Table scan on a (actual time=0.00337..0.0938 rows=1000 loops=1)
```

效率分析：

写法一通常最快，因为只需要排序后取 1 条，但不能返回并列第一。写法二语义最完整，能处理并列，但窗口函数需要对聚合后的学生平均分排序。`idx_sc205_sid_grade` 的主要影响点在 `SC205` 扫描：由 `Index scan on sc using PRIMARY` 或 `Table scan on sc` 变为 `Covering index scan on sc using idx_sc205_sid_grade`。因为平均分和计数只需要 `s_id` 与 `grade`，该覆盖索引减少了读取列和回表成本，所以写法一约 `11.4 ms -> 10.7 ms`，写法二约 `15.7 ms -> 14.2 ms`。写法三执行计划中 CTE 被优化器复用，显示的顶层时间很小，不适合单独作为索引效果判断依据。

查询 4：课程选课人数和平均分排名

该查询用于分析课程维度的热门程度和成绩情况。

写法一：直接按课程分组统计。

SQL

```
EXPLAIN ANALYZE
SELECT c.c_id, c.c_name, COUNT(*) AS student_count, AVG(sc.grade) AS avg_grade
FROM C205 c
JOIN SC205 sc ON sc.c_id = c.c_id
GROUP BY c.c_id, c.c_name
ORDER BY student_count DESC, avg_grade DESC
LIMIT 20;
```

写法二：先在 **SC205** 上按课程聚合，再连接 **C205**。

SQL

```
EXPLAIN ANALYZE
SELECT c.c_id, c.c_name, t.student_count, t.avg_grade
FROM C205 c
JOIN (
    SELECT c_id, COUNT(*) AS student_count, AVG(grade) AS avg_grade
    FROM SC205
    GROUP BY c_id
) t ON t.c_id = c.c_id
ORDER BY t.student_count DESC, t.avg_grade DESC
LIMIT 20;
```

写法三：只统计有效成绩的课程平均分。

SQL

```
EXPLAIN ANALYZE
SELECT c.c_id, c.c_name, t.student_count, t.avg_grade
FROM C205 c
JOIN (
    SELECT c_id, COUNT(grade) AS student_count, AVG(grade) AS avg_grade
    FROM SC205
    WHERE grade IS NOT NULL
    GROUP BY c_id
) t ON t.c_id = c.c_id
ORDER BY t.student_count DESC, t.avg_grade DESC
LIMIT 20;
```

优化索引：

SQL

```
CREATE INDEX idx_sc205_cid_grade ON SC205 (c_id, grade);
```

未建立 **idx_sc205_cid_grade** 时，三种写法的关键执行计划如下：

```

-- 写法一
-> Limit: 20 row(s) (actual time=55.4..55.4 rows=20 loops=1)
    -> Sort: student_count DESC, avg_grade DESC
        -> Aggregate using temporary table (actual time=55.3..55.3 rows=100 loops=1)
            -> Nested loop inner join (actual time=0.049..28.8 rows=19800 loops=1)
                -> Table scan on sc (actual time=0.0353..6 rows=19800 loops=1)
                    -> Single-row index lookup on c using PRIMARY

-- 写法二
-> Limit: 20 row(s) (actual time=35..35.1 rows=20 loops=1)
    -> Sort: t.student_count DESC, t.avg_grade DESC
        -> Materialize (actual time=35..35 rows=100 loops=1)
            -> Group aggregate (actual time=0.525..34.8 rows=100 loops=1)
                -> Index scan on SC205 using fk_sc205_c205
                    (actual time=0.272..30.7 rows=19800 loops=1)

-- 写法三
-> Limit: 20 row(s) (actual time=35.6..35.6 rows=20 loops=1)
    -> Sort: t.student_count DESC, t.avg_grade DESC
        -> Materialize (actual time=35.5..35.5 rows=100 loops=1)
            -> Group aggregate (actual time=0.468..35.4 rows=100 loops=1)
                -> Index scan on SC205 using fk_sc205_c205
                    (actual time=0.204..30 rows=19800 loops=1)

```

建立 `idx_sc205_cid_grade` 后，三种写法的关键执行计划如下：

```

-- 写法一
-> Limit: 20 row(s) (actual time=28.3..28.3 rows=20 loops=1)
    -> Sort: student_count DESC, avg_grade DESC
        -> Aggregate using temporary table (actual time=28.2..28.2 rows=100 loops=1)
            -> Nested loop inner join (actual time=0.0834..8.52 rows=19800 loops=1)
                -> Table scan on c (actual time=0.0368..0.064 rows=100 loops=1)
                    -> Covering index lookup on sc using idx_sc205_cid_grade
                        (c_id=c.c_id) (actual time=0.0294..0.0741 rows=198 loops=100)

-- 写法二
-> Limit: 20 row(s) (actual time=8.39..8.42 rows=20 loops=1)
    -> Sort: t.student_count DESC, t.avg_grade DESC
        -> Materialize (actual time=8.32..8.32 rows=100 loops=1)
            -> Group aggregate (actual time=0.108..8.25 rows=100 loops=1)
                -> Covering index scan on SC205 using idx_sc205_cid_grade
                    (actual time=0.0312..4.2 rows=19800 loops=1)

-- 写法三
-> Limit: 20 row(s) (actual time=9.63..9.66 rows=20 loops=1)
    -> Sort: t.student_count DESC, t.avg_grade DESC
        -> Materialize (actual time=9.56..9.56 rows=100 loops=1)
            -> Group aggregate (actual time=0.137..9.48 rows=100 loops=1)
                -> Covering index scan on SC205 using idx_sc205_cid_grade
                    (actual time=0.032..4.25 rows=19800 loops=1)

```

效率分析：

SC205 是最大表，课程统计主要耗时在对 **SC205** 的扫描和分组。**idx_sc205_cid_grade** 的影响最明显：写法一由先扫描 **SC205** 再回表查 **C205**，变成先扫描 100 行 **C205**，再按 **c_id** 对 **SC205** 做覆盖索引查找，耗时约 **55.4 ms -> 28.3 ms**。写法二和写法三由 **fk_sc205_c205(c_id)** 单列索引扫描变为 **idx_sc205_cid_grade(c_id, grade)** 覆盖索引扫描，**grade** 已在索引中，计算 **AVG(grade)** 不需要再读取完整行，因此分别约 **35.0 ms -> 8.39 ms**、**35.6 ms -> 9.63 ms**。由于 **student_count** 和 **avg_grade** 是聚合后产生的值，索引不能直接消除最后的排序，但能显著降低分组聚合前的扫描成本。

查询效率测试 SQL

可以通过下列 SQL 记录各查询执行前后的索引情况和执行计划：


```
SHOW INDEX FROM SC205;
```

```
EXPLAIN ANALYZE
```

```
SELECT s.s_id, s.s_name, SUM(c.credit) AS total_credit
FROM S205 s
JOIN SC205 sc ON sc.s_id = s.s_id
JOIN C205 c ON c.c_id = sc.c_id
GROUP BY s.s_id, s.s_name;
```

```
EXPLAIN ANALYZE
```

```
SELECT s.s_id, s.s_name, t.total_credit
FROM S205 s
JOIN (
    SELECT sc.s_id, SUM(c.credit) AS total_credit
    FROM SC205 sc
    JOIN C205 c ON c.c_id = sc.c_id
    GROUP BY sc.s_id
) t ON t.s_id = s.s_id;
```

六.触发器记录删除数据

本题要求设计触发器，实现删除 `sc205` 表数据时自动记录被删除记录。备份表命名为 `bk205`。题目中的学号、课程号字段在实际 SQL 中统一使用 `s_id`、`c_id` 表示，避免字段名中出现特殊符号带来的转义问题；`ddate` 记录删除发生的精确时间。

1. 创建备份表

```
drop table if exists bk205;
```

```
create table bk205 (
    s_id char(10) not null comment '被删除选课记录的学号',
    c_id varchar(10) not null comment '被删除选课记录的课程号',
    grade decimal(4,1) null comment '被删除选课记录的成绩',
    ddate datetime(6) not null comment '删除时间',
    primary key (s_id, c_id, ddate)
);
```

这里使用更适合 SQL 书写的 `s_id`、`c_id` 作为字段名。

2. 创建删除触发器

```
SQL

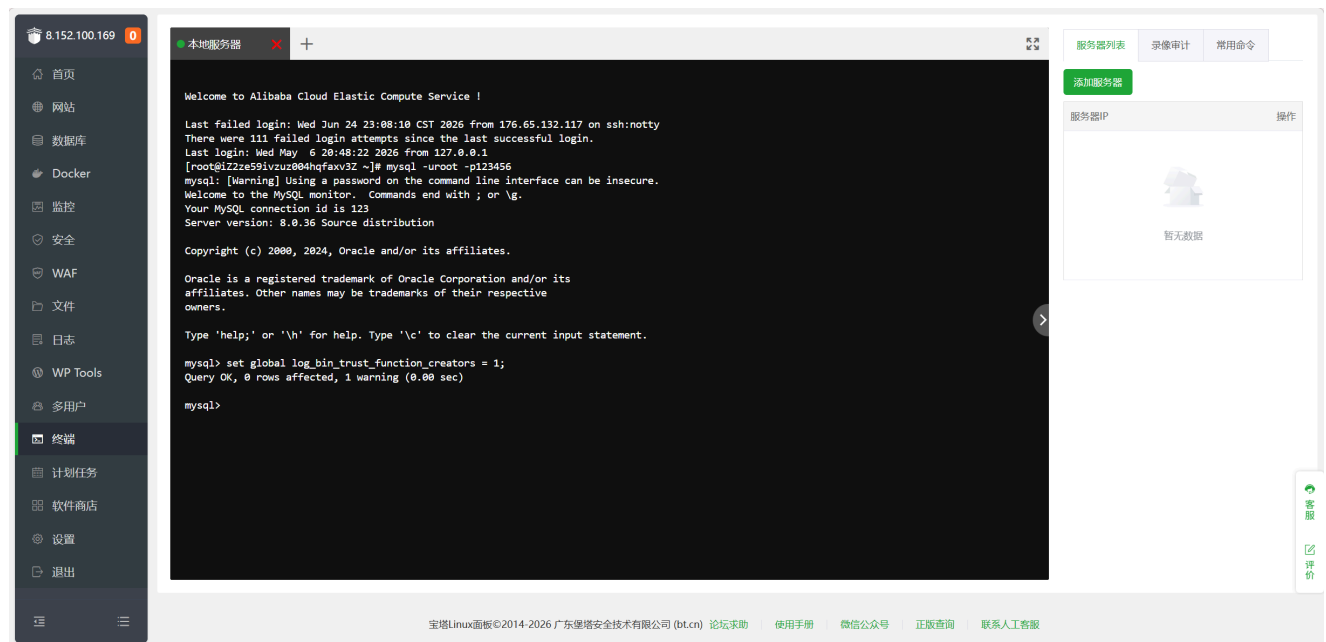
drop trigger if exists trg_sc205_delete_backup;

delimiter $$

create trigger trg_sc205_delete_backup
before delete on sc205
for each row
begin
    insert into bk205 (s_id, c_id, grade, ddate)
    values (old.s_id, old.c_id, old.grade, now(6));
end$$

delimiter ;
```

第一次创建触发器报错了,发现是权限的问题



到云端面板加上了权限

3. 随机删除 100 条成绩为 null 的选课记录

先检查 **sc205** 表中成绩为 **null** 的候选记录数量:

```
SQL

select count(*) as null_grade_count
from sc205
where grade is null;
```

	null_grade_count
1	633

随机选取 100 条成绩为 `null` 的记录并直接删除。触发器会在每一行被删除前自动把该记录写入 `bk205`：

SQL

```
delete from sc205
where grade is null
order by rand()
limit 100;
```

4. 结果查询语句

查询备份表中由触发器写入的记录数量：

SQL

```
select count(*) as backup_rows
from bk205;

select * from bk205;
```

	backup_rows
1	100

	s_id	c_id	grade	ddate
1	2224115857	LITE303721	<null>	2026-06-30 22:35:40.494873
2	2224121514	VENG431401	<null>	2026-06-30 22:35:40.494873
3	2225123485	BESE400222	<null>	2026-06-30 22:35:40.494873
4	2225213826	ACAU420608	<null>	2026-06-30 22:35:40.494873
5	2226114099	CORE100106	<null>	2026-06-30 22:35:40.494873
6	2226114164	INFT533305	<null>	2026-06-30 22:35:40.494873
7	2226114674	BIME413013	<null>	2026-06-30 22:35:40.494873
8	2226124426	ELEC427204	<null>	2026-06-30 22:35:40.494873
9	2232223025	INFT533305	<null>	2026-06-30 22:35:40.494873